

العمليات المنطقية (البوليانية)

10

هناك قيمتان فقط لنوع **البيانات المنطقية**، وهما True و False. تُستخدم الحسابات

المنطقية في البرمجة لإجراء الموازنات، والتحكم في مسار البرنامج.

تمثل القيم المنطقية قيم الحقيقة (truth values) في علم المنطق في الرياضيات، وتُكتب

القيمتان True و False دائماً بالحرفين الكبيرين T و F على التوالي، لأنَّهما قيمتان خاصتان

في بايثون. سنتعرف في هذا الفصل على العمليات المنطقية في بايثون، بما في ذلك الموازنة

المنطقية، والعوامل المنطقية، وجداول الحقيقة.

1. عامل الموازنة

في البرمجة، تُستخدم عاملات الموازنة (comparison operators) للموازنة بين القيم،

وتعيد إحدى القيمتين المنطقتين True و False. يوضح الجدول أدناه عاملات

الموازنة المنطقية:

الشرح	العامل
يساوي	==
يخالف	!=
أصغر من	>
أكبر من	<
أصغر من أو يساوي	>=
أكبر من أو يساوي	<=

لفهم كيفية عمل هذه العلامات، سنستخدم المتغيرين التاليين:

```
x = 5
y = 8
```

في هذا المثال، لَمَّا كان x يساوي 5، فهو أصغر من y ذي القيمة 8.

باستخدام هذين المتغيرين والقيم المرتبطة بهما، سنجرّب أحد العلامات من الجدول أعلاه.

سنطلب من بايثون أن تطبع ناتج عملية الموازنة، إمّا True أو False. لتوضيح المثال أكثر،

سنطبع سلسلة نصية لتوضيح ما جرى تقييمه.

```
x = 5
y = 8

print("x == y:", x == y)
print("x != y:", x != y)
print("x < y:", x < y)
print("x > y:", x > y)
print("x <= y:", x <= y)
print("x >= y:", x >= y)
```

والمخرجات هي:

```
x == y: False
x != y: True
x < y: True
x > y: False
x <= y: True
x >= y: False
```

باتباع المنطق الرياضي، في كل من التعبيرات المذكورة أعلاه، هذه نتيجة الموازنات:

- 5 (قيمة x) تساوي 8 (قيمة y)؟ False، خطأ

- 5 تخالف 8؟ True، صحيح
- 5 أصغر من 8؟ True، صحيح
- 5 أكبر من 8؟ False، خطأ
- 5 أصغر من أو يساوي 8؟ True، صحيح
- 5 ليس أصغر من أو يساوي 8؟ False، خطأ

رغم استخدامنا للأعداد الصحيحة هنا، إلا أنه بإمكاننا استبدال الأعداد العشرية بها. يمكن أيضًا استخدام السلاسل النصية مع المعاملات المنطقية. وهي حساسة لحالة الأحرف، ما لم تستخدم تابعًا إضافيًا للسلاسل النصية، ويوضح المثال التالي كيفية موازنة السلاسل النصية:

```
Sammy = "Sammy"
sammy = "sammy"

print("Sammy == sammy: ", Sammy == sammy)      # Sammy == sammy:
False
```

السلسلة "Sammy" أعلاه لا تساوي السلسلة النصية "sammy"، لأنهما ليستا متماثلتين تمامًا؛ فإحدهما تبدأ بحرف كبير S، والأخرى بحرف صغير s. ولكن لو أضفنا متغيرًا آخر قيمته "Sammy"، فستكونان متساويتين:

```
Sammy = "Sammy"
sammy = "sammy"
also_Sammy = "Sammy"
```

```
print("Sammy == sammy: ", Sammy == sammy)
# Sammy == sammy: False
print("Sammy == also_Sammy", Sammy == also_Sammy)
# Sammy == also_Sammy: True
```

يمكنك أيضًا استخدام علامات الموازنة الأخرى، بما في ذلك < و > لموازنة سلسلتين نصيتين. ستوازن بايثون هذه السلاسل النصية بحسب الترتيب المعجمي في نظام محارف ASCII. ويمكنك أيضًا تقييم القيم المنطقية باستخدام علامات الموازنة:

```
t = True
f = False

print("t != f: ", t != f)    # t != f: True
```

تبيّن الشيفرة البرمجية أعلاه أنَّ True لا تساوي False.

لاحظ الفرق بين العاملين = و ==.

```
x = y    # إسناد قيمة y إلى x
x == y   # نتحقق مما إذا كان y و x متساويين
```

الأول =، هو عامل الإسناد (assignment operator)، والذي سيحدّد قيمة أحد المتغيّرين، ويجعلها مساوية لقيمة الآخر. الثاني ==، وهو عامل الموازنة الذي سيحدّد ما إذا كانت القيمتان متساويتين.

2. العوامل المنطقية

هناك ثلاثة عوامل منطقية (Boolean operators) تُستخدم لموازنة القيم. وتعيد إمّا True أو False. هذه العوامل هي، and (و)، or (أو)، و not (النفي)، وقد عرّفناها في الجدول أدناه.

الصياغة	الشرح	العامل
x and y	إن كان كلا التعبيرين صحيحين True	and
x or y	إن كان أحد التعبيرين على الأقل True صحيحًا	or
not x	إن كان التعبير خطأ True	not

عادةً ما تُستخدم العوامل المنطقية لتقييم ما إذا كان تعبيران منطقيان صحيحين أم لا. على سبيل المثال، يمكن استخدامها لتحديد ما إذا كان الطالب قد نجح، وأنه مُسجل في الفصل، وإذا كانت كلتا الحالتان صحيحتين، فسيُضاف الطالب إلى سجل النظام. مثال آخر هو تحديد ما إذا كان المستخدم عميلًا نشطًا لمتجر إلكتروني استنادًا إلى ما إذا كان لديه رصيد في المتجر، أو أنه اشترى خلال الأشهر الستة الماضية.

لفهم كيفية عمل العوامل المنطقية، دعنا نقيّم التعابير الثلاث التالية:

```
print((9 > 7) and (2 < 4)) # كلا التعبيرين صحيحان
print((8 == 8) or (6 != 6)) # أحد التعبيرين صحيح
print(not(3 <= 1)) # التعبير الأصلي خطأ
```

والمخرجات هي:

```
True
True
True
```

في الحالة الأولى، `print((9 > 7) and (2 < 4))`، ينبغي أن يكون كلا التعبيرين

`9 > 7` و `2 < 4` صحيحين لتكون النتيجة `True` لأنَّ المعامل `and` مُستخدَم.

في الحالة الثانية، `print((8 == 8) or (6 != 6))`، بما أنَّ قيمة `8 == 8` قُيِّمت

إلى `True`، فلا يهم ما يقيَّم إليه التعبير `6 != 6`. لكن لو استخدمنا العامل `and`، لُقِّمَّ التعبير إلى القيمة `False`.

في الحالة الثالثة، `print(not(3 <= 1))`، ينفي العامل `not` القيمة `False` التي تعيدها

ناتج العملية المنطقية `3 <= 1`.

في المثال التالي، سنستبدل بالأعداد العشرية أعدادًا صحيحة:

```
print((-0.2 > 1.4) and (0.8 < 3.1)) # أحد التعبيرين خطأ
print((7.5 == 8.9) or (9.2 != 9.2)) # كلا التعبيرين خطأ
print(not(-5.7 <= 0.3))           # التعبير الأصلي صحيح
```

في المثال أعلاه:

- `and`: يجب أن يكون واحد على الأقل من التعبيرين خطأ ليعيد القيمة `False`.
- `or`: يجب أن يكون كلا التعبيرين خطأ لتعيد القيمة `False`.
- `not`: يجب أن يكون التعبير المرافق له صحيحًا حتى يعيد القيمة `False`.

إذا لم تكن النتائج أعلاه واضحة، فسنعرض بعض جداول الحقيقة أدناه لتفهم الأمر

فهنا أفضل.

يمكنك أيضًا كتابة عبارات مُركَّبة باستخدام `and`، و `or`، و `not`:

```
not((-0.2 > 1.4) and ((0.8 < 3.1) or (0.1 == 0.1)))
```

التعبير الداخلي: $(0.1 == 0.1) \text{ or } (3.1 > 0.8)$ يعيد القيمة True، لأنَّ كلا التعبيرين الرياضييين محققان، أي يعيدان True.

الآن، نأخذ القيمة المُعادة True ونجمعها مع التعبير المنطقي التالي:

```
(-0.2 > 1.4) and (True)
```

هذا المثال يعيد False، لأنَّ التعبير $-0.2 > 1.4$ يقيّم إلى القيمة False، و $(False) \text{ and } (True)$ ينتج عنه القيمة False.

أخيرًا، يعيد التعبير الخارجي: $\text{not } (False)$ القيمة True، وبالتالي ستكون القيمة النهائية المعادة هي:

```
True
```

3. جداول الحقيقة (Truth Tables)

المنطق مجال واسع، وسنكتفي في هذا الفصل ببعض الأفكار المهمّة التي يمكن أن تساعدك على تحسين طريقة تفكيرك وخوارزمياتك.

فيما يلي جداول الحقيقة لمعامل الموازنة $==$ ، والمعاملات المنطقية and ، و or ، و not . من المفيد أن تحفظ كَيْفِيَّة عملها، فذلك سيجعلك أسرع في اتخاذ القرارات أثناء كتابة الشيفرات البرمجية.

١. جدول الحقيقة الخاص بالعامل ==

القيمة المُعادة	y	==	x
True	True	==	True
False	False	==	True
False	True	==	False
True	False	==	False

ب. جدول الحقيقة الخاص بالعامل AND

القيمة المُعادة	y	and	x
True	True	and	True
False	False	and	True
False	True	and	False
False	False	and	False

ج. جدول الحقيقة الخاص بالعامل OR

القيمة المُعادة	y	or	x
True	True	or	True
True	False	or	True
True	True	or	False
False	False	or	False

د. جدول الحقيقة الخاص بالعمل NOT

القيمة المُعادَة	not	x
False	not	True
True	not	False

تُستخدَم جداول الحقيقة في المنطق الرياضي كثيرًا، وهي مفيدة، ويجب حفظها ووضوعها في الحسبان عند إنشاء الخوارزميات البرمجية.

4. استعمال المنطق للتحكم في مسار البرنامج

للتحكم في مسار ونتائج البرنامج عبر التعليمات الشرطية (flow control statements)، يمكننا استخدام «شرط» (condition) متبوعًا «بعبارة برمجية» (clause).

يُقيَّم الشرط بإحدى القيمتين True أو False، و تُستخدَم تلك القيمة في اتخاذ قرار في البرنامج. أمَّا العبارة (clause) فهي الكتلة البرمجية التي تعقب الشرط وتحدّد نتيجة البرنامج وما ينبغي فعله في حال تحقق الشرط أو عدمه.

تُظهر الشيفرة أدناه مثالًا على معاملات الموازنة التي تعمل مع العبارات الشرطية للتحكم

في مسار البرنامج:

```

if grade >= 65:           # شرط
    print("Passing grade") # بند

else:
    print("Failing grade")

```

سيحدّد هذا البرنامج ما إذا كان الطالب سينجح أم يرسب. في حال كانت الدرجة التي حصل عليها الطالب تساوي 83 مثلاً، تقييم العبارة الأولى سيكون True، وسيطّبع النص "Passing grade". أمّا إن كانت درجة الطالب هي 59، فتقييم العبارة الأولى سيكون False، وبالتالي سينتقل البرنامج لتنفيذ التعليمة المرتبطة بالفرع else، أي سيطّبع "Failing grade".

يمكن تقييم كل كائنات بايثون بإحدى القيمتين True أو False، لذلك **يوصي الدليل 8 PEP** بعدم موازنة كائن بإحدى القيمتين True أو False، لأنّ ذلك قد يؤدي إلى إعادة قيم منطقية غير متوقّعة. على سبيل المثال، عليك تجنب استخدام مثل هذا التعبير `True == sammy` في برامجك.

تساعد العوامل المنطقية على صياغة شروط يمكن استخدامها لتحديد النتيجة النهائية للبرنامج من خلال التحكم في مسار التنفيذ.

5. خلاصة الفصل

ألقينا في هذا الفصل نظرة على الموازنات والعوامل المنطقية، بالإضافة إلى جداول الحقيقة، وكيفية استخدام القيم المنطقية للتحكم في مسار البرنامج.